

# hw3

## 李毅PB22051031

### 1.计算下图的 DFT, DCT, Hadamard 变换和Haar变换

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |

代入DFT公式:

$$F(u, v) = \sum_{x=0}^3 \sum_{y=0}^3 f(x, y) e^{-j2\pi(ux/4+vy/4)}$$

得DFT为:

$$\begin{bmatrix} 8 + 0j & -4 - 4j & 0 & -4 + 4j \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

代入DCT公式:

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^3 \sum_{y=0}^3 f(x, y) \cos \left[ \frac{(2x+1)u\pi}{8} \right] \cos \left[ \frac{(2y+1)v\pi}{8} \right]$$
$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{4}} & \text{if } u = 0 \\ \sqrt{\frac{2}{4}} & \text{if } u \neq 0 \end{cases}$$
$$\alpha(v) = \begin{cases} \sqrt{\frac{1}{4}} & \text{if } v = 0 \\ \sqrt{\frac{2}{4}} & \text{if } v \neq 0 \end{cases}$$

得DCT为:

$$\begin{bmatrix} 2 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Hadamard 变换矩阵为:

$$\frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

由  $T = AFA^T$  得结果为:

$$\begin{bmatrix} 2 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Harr变换矩阵为:

$$\frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ \sqrt{2} & -\sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{2} & -\sqrt{2} \end{bmatrix}$$

由  $T = AFA^T$  得结果为:

$$\begin{bmatrix} 2 & 0 & -\sqrt{2} & \sqrt{2} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

**2. 设有一组64\*64的图像， 它们的协方差矩阵式单位矩阵.如果只使用一半的原始特征值计算重建图像,那么原始图像和重建图像间的均方误差是多少?**

协方差矩阵为 (64\*64) \* (64\*64) 大小。所有特征值为1, 故:

$$e_{ms} = \sum_{i=1}^{64 \times 64} \lambda_i - \sum_{i=1}^{64 \times 64 / 2} \lambda_i = 2048$$

**3. 编程实现lena.bmp的离散Fourier变换和离散余弦变换，并显示频谱图像。**

```
1 import numpy as np
2 import cv2
3
4 def DFT2D(matrix):
5     M, N = matrix.shape
6     u = np.arange(M).reshape(-1, 1)
7     v = np.arange(N).reshape(-1, 1)
8     x = np.arange(M).reshape(1, -1)
9     y = np.arange(N).reshape(1, -1)
10
11     w_M = np.exp(-2j * np.pi * u @ x / M)
```

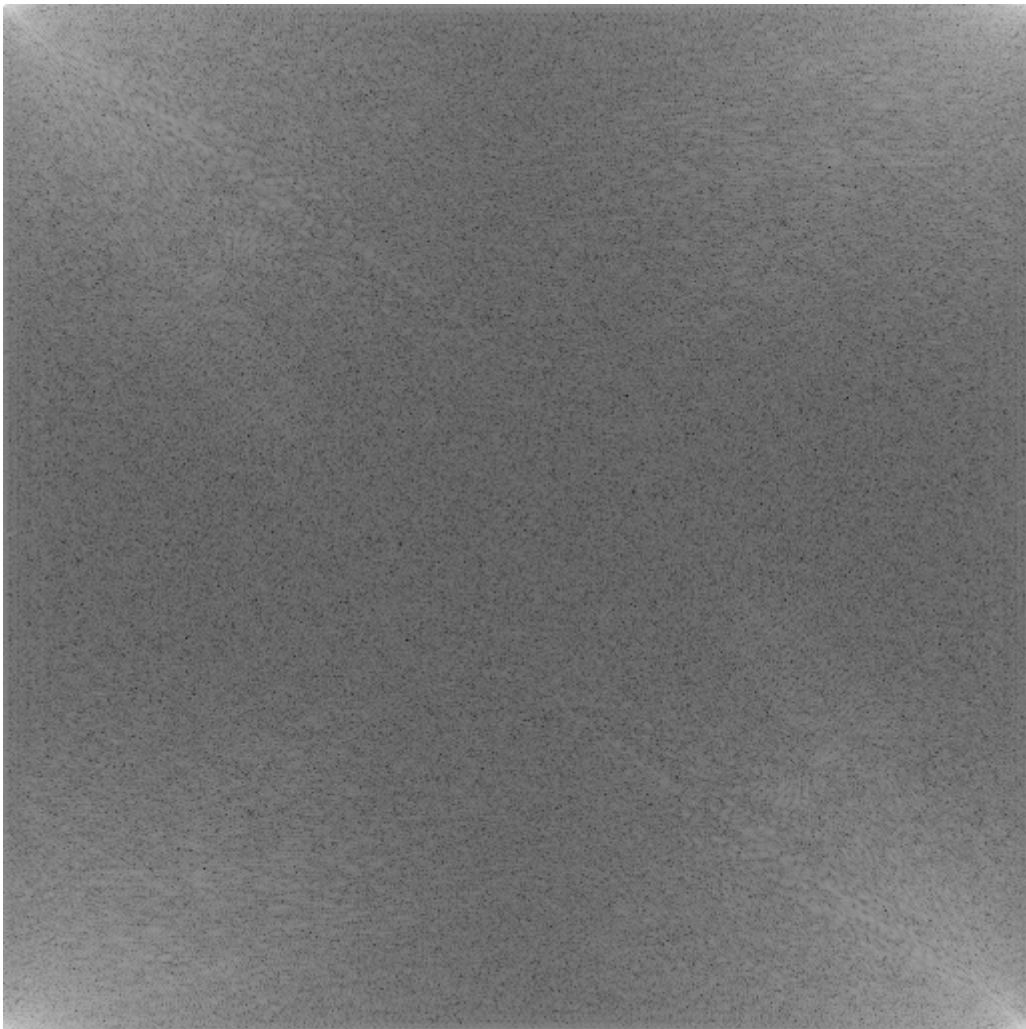
```

12     W_N = np.exp(-2j * np.pi * v @ y / N)
13
14     return W_M @ matrix @ W_N
15
16 def DCT2D(matrix):
17     M, N = matrix.shape
18     u = np.arange(M).reshape(-1, 1)
19     v = np.arange(N).reshape(-1, 1)
20     x = np.arange(M).reshape(1, -1)
21     y = np.arange(N).reshape(1, -1)
22
23     alpha_u = np.sqrt(1 / M) * (u == 0) + np.sqrt(2 / M) * (u != 0)
24     alpha_v = np.sqrt(1 / N) * (v == 0) + np.sqrt(2 / N) * (v != 0)
25
26     cos_u = np.cos((2 * x + 1) * u * np.pi / (2 * M))
27     cos_v = np.cos((2 * y + 1) * v * np.pi / (2 * N))
28
29     return (alpha_u * cos_u) @ matrix @ (alpha_v * cos_v).T
30
31 image = cv2.imread('lena.bmp', cv2.IMREAD_GRAYSCALE)
32
33 # 计算二维DFT并保存结果
34 dft_result = DFT2D(image)
35 dft_magnitude = np.log(np.abs(dft_result) + 1)
36 cv2.imwrite('Lena_dft.bmp', (dft_magnitude / dft_magnitude.max() *
37 255).astype(np.uint8))
38
39 # 计算二维DCT并保存结果
40 dct_result = DCT2D(image)
41 dct_magnitude = np.log(np.abs(dct_result) + 1)
42 cv2.imwrite('Lena_dct.bmp', (dct_magnitude / dct_magnitude.max() *
43 255).astype(np.uint8))
44
45 # 平移图像后计算DFT并保存
46 shifted_image = image * np.fromfunction(lambda i, j: (-1)**(i + j),
47 image.shape)
48 dft_result2 = DFT2D(shifted_image)
49 dft_magnitude2 = np.log(np.abs(dft_result2) + 1)
50 cv2.imwrite('Lena_dft2.bmp', (dft_magnitude2 / dft_magnitude2.max() *
51 255).astype(np.uint8))

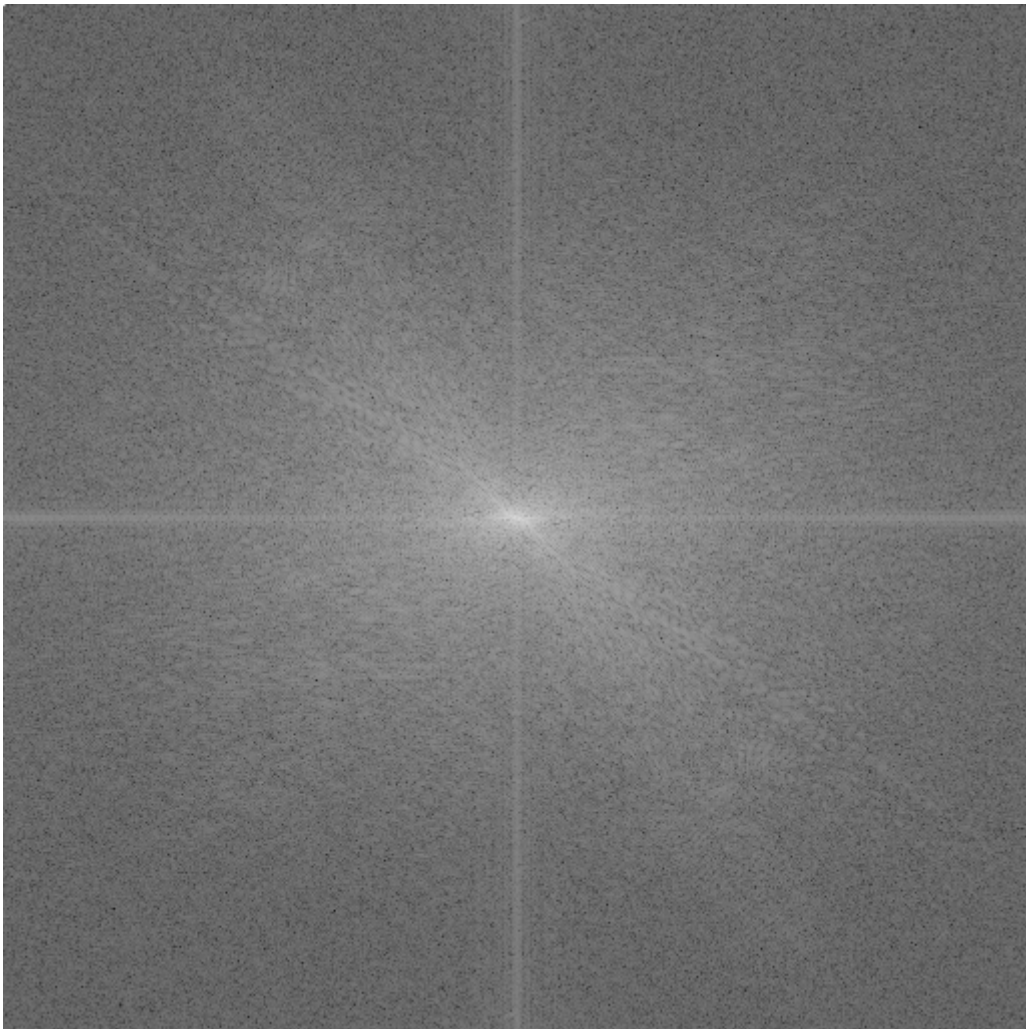
```

为体现区别便于观察，对幅度进行了取对数处理。直接使用循环DFT的算法复杂度为 $O(n^4)$ ，运行速度太慢了，这也不符合实际操作逻辑。改用numpy库进行**矩阵运算**可以大大加速这一过程。

离散Fourier变换如下：



x, y方向均平移 $N/2$ 个单位后得到结果如下：



离散余弦变换如下：

